

Onboard Deep Reinforcement Learning: Deployment and Testing for CubeSat Attitude Control [†]

Sajjad Zahedi ¹, Jafar Roshanian ^{1,*}, Mehran Mirshams ² and Krasin Georgiev ^{3,*} 

¹ Intelligent Control Systems Institute, K. N. Toosi University, Tehran 19697-64499, Iran; 1sajjadzahedi1@gmail.com

² Space Research Laboratory, K. N. Toosi University, Tehran 16569-83911, Iran; mirshams@kntu.ac.ir

³ National Centre of Excellence “Mechatronics and Clean Technologies”, Technical University of Sofia, 1000 Sofia, Bulgaria

* Correspondence: roshanian@kntu.ac.ir (J.R.); krasin@tu-sofia.bg (K.G.)

[†] Presented at the 17th International Scientific Conference on Aerospace, Automotive, and Railway Engineering (BulTrans-2025), Sozopol, Bulgaria, 10–13 September 2025.

Abstract

Recent progress in Reinforcement Learning (RL), especially deep RL, has created new possibilities for autonomous control in complex and uncertain environments. This study explores these possibilities through a practical approach, implementing an RL agent on a custom-built CubeSat. The CubeSat, equipped with a reaction wheel for active attitude control, serves as a physical testbed for validating RL-based strategies. To mimic space-like conditions, the CubeSat was placed on a custom air-bearing platform that allows near-frictionless rotation along a single axis, simulating microgravity. Unlike simulation-only research, this work showcases real-time hardware-level implementation of a Double Deep Q-Network (DDQN) controller. The DDQN agent receives real system state data and outputs control commands to orient the CubeSat via its reaction wheel. For comparison, a traditional PID controller was also tested under identical conditions. Both controllers were evaluated based on response time, accuracy, and resilience to disturbances. The DDQN outperformed the PID, showing better adaptability and control. This research demonstrates the successful integration of RL into real aerospace hardware, bridging the gap between theoretical algorithms and practical space applications through a hands-on CubeSat platform.

Keywords: CubeSat; reinforcement learning; Intelligent Control; hardware-in-loop

1. Introduction

As technology advances, the need for a more accurate understanding of the world and the development of advanced machines has been increasing. However, along with these advancements, control challenges have also become more complex. For instance, satellites often face unpredictable challenges during their missions, such as sudden changes in Earth’s magnetic field, which can cause deviations in their orientation. Additionally, the effects of Earth’s geopotential and the Moon’s asymmetric gravity can impact both their orientation and orbital stability. In such situations, there is a growing demand for controllers capable of adapting to unknown environments and unpredictable changes.

The attitude control of CubeSats is one of the popular topics in satellite control. Researchers have developed various methods for this purpose. Some researchers have used PID controllers [1–3] due to their simplicity and accessibility. However, tuning PID



Academic Editor: Svetla Stoilova

Published: 20 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

controllers in systems with changing parameters, such as inertia momentum, is difficult, and in real situations, they cannot be re-tuned dynamically. Fuzzy control methods have been used by a group of researchers [4], but in unpredictable situations, finding their optimal point is complex and time-consuming. Combining fuzzy control with algorithms like genetic algorithms can improve its performance [5]. However, this combination requires generating all possible solutions and evaluating each of them, which makes it unsuitable for systems that require immediate reactions.

Other methods, such as the Singular Nonlinear Controller (SNC) and the Quaternion-Based Nonlinear Controller (QBNC) [6], have strong nonlinear properties that allow them to effectively handle noise and maintain good performance at critical points. However, these methods require an exact system model, and any changes in the system can cause disturbances in their performance. On the other hand, Reinforcement Learning (RL) is a promising approach for satellite control, as it is model-free and can achieve better performance by continuously exploring and exploiting the environment [7].

Among RL algorithms, Deep Deterministic Policy Gradient (DDPG) is one of the options for controlling continuous systems and can be used in satellite attitude control [8]. This algorithm uses a neural network to learn control policies and evaluate the environment. Additionally, the Proximal Policy Optimization (PPO) algorithm, which makes policy changes gradually, is a suitable choice for sensitive systems like CubeSats [9]. Some researchers have used the Hierarchical Deep Reinforcement Learning (HDRL) algorithm, which employs a hierarchical structure and deep neural networks [10]. This method helps reduce search complexity and prevents excessive oscillations. However, this approach requires defining exact subtasks, which makes its implementation challenging. In addition to attitude control, RL has been used in other areas, such as energy management and task scheduling for CubeSats [11,12].

However, there is a gap between the theoretical development and practical implementation of RL in space systems (such as satellites). Moreover, only a limited number of studies have implemented RL in real-world space systems. Additionally, the Double Deep Q-Network (DDQN) algorithm, which combines neural networks with Q-learning, has a strong capability in learning control commands and optimizing decision-making. Compared to methods such as HDRL, DDQN requires less computation and, with experience replay, can store past data and utilize it for improved decision-making, eliminating the need for hierarchical modeling.

This study employs the DDQN algorithm for CubeSat attitude control using a reaction wheel. It is then implemented on a real CubeSat in a laboratory environment using a disturbance generator stand and an air-bearing platform, followed by its evaluation. Finally, the performance of DDQN is compared with a PID controller to establish a specific criterion for evaluating its advantages and disadvantages.

2. CubeSat Design and Dynamic Modeling on a Test Stand

An appropriate dynamic model of a CubeSat was developed using a CubeSat assembly constructed in [13], which was further developed for the deep learning system with its structure being modified. The unit was designed with dimensions of 12 cm × 12 cm × 12 cm and a mass of 0.67 kg, incorporating a reaction wheel as its actuator (Figure 1). The reaction wheel is composed of two materials: an inner core made of aluminum and an outer rim made of brass. The reaction wheel's performance is enhanced by the higher density of brass through increased moment of inertia.



Figure 1. Reaction wheel.

The diagrams and hardware setup will be detailed in the Real-Time Implementation section. The CubeSat is mounted on a disk, with its position secured by four mechanical clamps. These clamps not only prevent the CubeSat from slipping but also allow for fine adjustments of the center of mass relative to the disk's central axis by attaching nuts to them.

To introduce disturbances into the system, four masses are suspended symmetrically around the disk. These masses destabilize the system by generating torque as the disk rotates, leading to undesired angular deviations and affecting the overall system dynamics. Beneath the disk, a hemispherical structure is mounted on an air-bearing platform. When air is pumped into the bearing, the friction between the disk and the platform is effectively eliminated, enabling near-frictionless rotation (Figure 2).

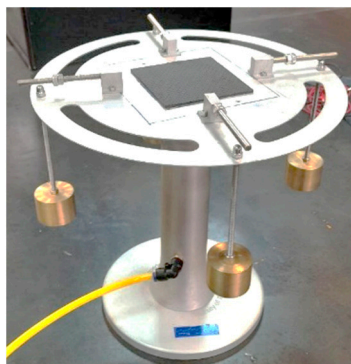


Figure 2. CubeSat.

Figure 3 shows the main CubeSat structure, and in Figure 4, the CubeSat is mounted on the test stand and ready to receive control commands.

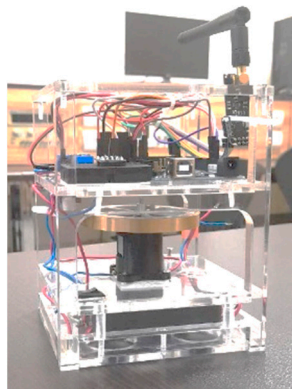


Figure 3. Test stand.

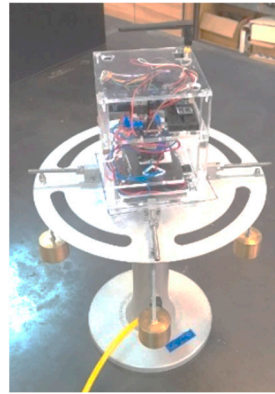


Figure 4. CubeSat on test stand.

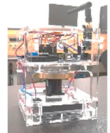
To enable simulation and gain a deeper understanding of the system's behavior, it is essential to formulate the dynamic equations. These equations describe the system as follows: when the reaction wheel rotates, it produces a reaction torque due to its rotational inertia, which in turn induces rotation of the disk. Furthermore, all disturbance torques generated by the test disk are consolidated into a single disturbance term in the dynamic model, which can later be estimated. The resulting dynamic equation is presented below (1):

$$J_{rw}\ddot{\theta}_{rw} = J_{CS}\ddot{\theta}_{CS} + M_d, \quad (1)$$

In the above equation, J_{rw} represents the moment of inertia of the reaction wheel and $\ddot{\theta}_{rw}$ represents angular acceleration. Similarly, J_{CS} represents moment of inertia of the system, $\ddot{\theta}_{CS}$ represents angular acceleration of the system, and M_d represents the disturbance moments. In this system, $\dot{\theta}_{rw}$ is input signal and θ_{CS} is output signal.

Calculating the moment of inertia of the system requires the mass of each component of the system (Table 1).

Table 1. Weight properties of the system.

Mass	Weight	Title 3
each bar from which a weight is hung.	20	
each weight that is hanging.	311	
Total disk (without CubeSat)	1884.4	
CubeSat	617.3	
Reaction wheel	68.2	

The reaction wheel's moment of inertia can be easily calculated and its value is:

$$J_{rw} = 7.597 \times 10^{-5} \text{ kg} \cdot \text{m}^2$$

For the system (CubeSat and test disk), the moment of inertia is calculated using (2):

$$J_{surface} = 4J_{rod} + 4J_{weight} + J_{plane} + J_{cubesat}, \quad (2)$$

In the above equation, 'rod' refers to the bars that hang the masses, 'weight' refers to the masses that are hanging from the disk, 'plane' represents the test disk, and 'cubesat' represents the main structure of the CubeSat.

Each part's moment of inertia around the central axis of the disk is calculated using the parallel axis theorem (3):

$$I = I_0 + M \cdot d^2 \quad (3)$$

In the above equation I_0 represents the moment of inertia around the central axis, M represents the mass in kg and d represents the distance between two axes. The moments of inertia for the remaining components are presented below:

$$J_{rod} = 3.920625 \times 10^{-4} \text{ kg} \cdot \text{m}^2,$$

$$J_{weight} = 6.1578 \times 10^{-3} \text{ kg} \cdot \text{m}^2$$

$$J_{plane} = 6.3495 \times 10^{-3} \text{ kg} \cdot \text{m}^2$$

$$J_{cube} = 1.506314 \times 10^{-3} \text{ kg} \cdot \text{m}^2$$

Finally, the moment of inertia of the system according to (2) is:

$$J_{surface} = 0.034055 \text{ kg} \cdot \text{m}^2$$

To evaluate the controller's performance under varying levels of disturbance, the disturbance moment—having no fixed or known value—is assigned randomly during simulations. This approach allows for observing the controller's response to a range of unpredictable external torques.

3. Methodology

3.1. DDQN

As mentioned in previous sections, this research employs the Double Deep Q-Network (DDQN) algorithm to control the system. This section describes the structure of this algorithm.

DDQN is an extension of the Deep Q-Network (DQN), which combines Q-learning with neural networks. DQN faces several challenges that led to the development of DDQN. One major issue in DQN is that it uses the same network for both action selection and evaluation, which can result in slower learning and convergence to a suboptimal policy.

To address this, DDQN introduces two separate neural networks: (1) Online Network—responsible for selecting actions and (2) Target Network—responsible for evaluating actions.

This separation helps to mitigate overestimation bias, leading to more stable learning. However, it is important to note that DDQN requires additional computational resources compared to DQN. In summary, DDQN improves upon DQN by decoupling action selection and evaluation, reducing overestimation bias, and enhancing the stability and reliability of learning.

As shown in Figure 5, the structure of DDQN operates as follows: The environment sends the current state, next state, action, and reward to memory. Additionally, it sends the next state to the online network. The target network retrieves the next state from memory, updates itself, and sends the results to the loss function. The loss function receives the reward from memory along with the target network's output, processes these values, and sends the updated information to the online network. The online network then updates itself and selects the next action, which is sent back to the environment.

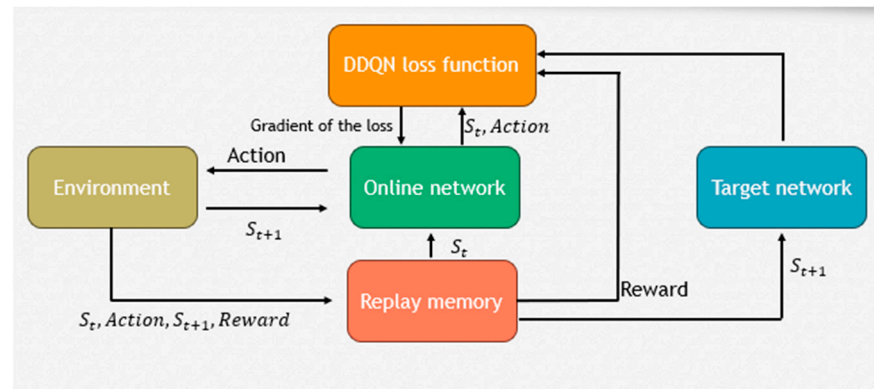


Figure 5. DDQN structure.

A neural network architecture was developed in MATLAB (version R2021b) to process the CubeSat's state and generate suitable control actions. The inputs to the network consist of the CubeSat's angular velocity and orientation, representing its dynamic state. The network starts with a feature input layer that matches the dimension of these input variables. This is followed by several fully connected layers with ReLU activation functions, allowing the model to learn nonlinear relationships within the data. The final fully connected layer outputs values that are compatible with the defined action space, making the network suitable for control tasks.

The replay memory used in this network consists of 50,000 units. Choosing an appropriate reward function is a crucial and influential challenge in designing a reinforcement learning (RL) agent. The reward function should be designed in a way that guides the agent toward the desired goal by maximizing the rewards received.

After testing multiple types of reward functions, (4) has been selected as the final reward function:

$$Reward = \begin{cases} |e_\theta| < 3^\circ \rightarrow Reward = 10 - |e_\theta| \\ else \rightarrow Reward = 0 \end{cases} \quad (4)$$

where e_θ is represented as (5):

$$e_\theta = (\theta_{des} - \theta)_{CS} \quad (5)$$

In the above equation, θ is current angle and θ_{des} is desired angle and CS is abbreviation for CubeSat. Agent should behavior to minimize the error function to maximize the reward function.

3.2. PID

The PID controller is one of the most widely used controllers for control dynamic systems. It consists of three components: (1) proportional (P) (2) integral (I) and (3) Derivative (D). this controller defined as (6):

$$u(t) = k_p e_\theta + k_i \int e_\theta dt + k_d \frac{de_\theta}{dt} \quad (6)$$

In the above equation e represents the error as mentioned in Equation (5) and u is input signal. One of the key challenges in PID is tuning its coefficients. There are several established methods for tuning the parameters of a PID controller, each with its own advantages depending on the characteristics of the system being controlled. Among the most widely used techniques are the Ziegler–Nichols and Cohen–Coon methods, both of which rely on empirical approaches to system modeling. However, they differ significantly in terms of implementation and the nature of the system response they produce.

The Ziegler–Nichols method is a heuristic tuning approach that does not require an explicit model of the system. It is typically applied by identifying the ultimate gain and oscillation period of the system in a closed-loop configuration. This method is particularly useful for systems that demand a fast response, although it often results in aggressive control behavior and noticeable oscillations, which may not be desirable in sensitive applications.

In contrast, the Cohen–Coon method is based on a first-order plus time delay (FOPTD) approximation of the system's open-loop step response. By fitting a mathematical model to the system's transient behavior, it provides more analytical tuning rules that tend to result in smoother responses and improved stability, particularly for systems with measurable time delays.

Given the sensitivity of the system addressed in this study—where excessive oscillations could lead to instability or physical damage—the Cohen–Coon method has been selected as the preferred tuning strategy. Its ability to produce a more stable and controlled response aligns better with the safety and performance requirements of the application.

4. Simulation

4.1. DDQN Control Simulation

The learning steps for the DDQN process use a dynamic model as a reference to evaluate performance and calculate rewards. All simulations are implemented in MATLAB scripts.

In the simulation environment, a feedforward neural network was employed to approximate the control policy for the CubeSat system. The network is designed to map the system's current state to an appropriate control action, effectively acting as a function approximator within the reinforcement learning framework.

The input to the network consists of two key state variables: the angular velocity and the orientation (attitude) of the CubeSat. These parameters capture the essential dynamic behavior of the system and are sufficient for defining its current configuration. By using these inputs, the network is capable of learning the underlying dynamics and generating control signals that stabilize or guide the CubeSat as required.

The architecture of the neural network comprises a sequence of fully connected layers interleaved with nonlinear activation functions (ReLU). These hidden layers serve to extract abstract features and model the complex, nonlinear relationships between the input state and the desired action. The depth and activation structure are carefully chosen to balance model expressiveness with training efficiency, ensuring that the network can generalize well across a variety of system states.

The final layer of the network maps the extracted features to the action space, ensuring compatibility with the control signals expected by the simulation environment. The output is typically continuous, representing values such as torque commands or normalized control signals, depending on the specifics of the actuator interface.

This neural network, when trained using reinforcement learning, continuously adjusts its internal weights based on the received reward signals. Over time, it learns to produce control actions that maximize performance criteria such as stability, responsiveness, or energy efficiency. The network's lightweight structure also makes it well-suited for potential deployment in embedded environments with limited computational resources.

Also, the Epsilon greedy values are provided in Table 2.

Table 2. Epsilon Options.

Epsilon	Epsilon Decay	Min Epsilon
0.9	0.005	0.01

These values enable the system to perform more exploration initially. Over time, as the agent gains experience, the exploration rate decreases, allowing the agent to rely more on its learned experience. However, the exploration rate never falls below 0.01, ensuring that the system continues to search for better rewards at all times.

To prevent over-gradient issues, the L2 norm method has been applied. Additionally, the optimization algorithm used for the reinforcement learning (RL) agent is ADAM. The learning rate is set to 0.01, and further details are provided in Table 3.

Table 3. Other Agent options.

Option	Value
Mini Batch Size	64
Target smooth factor	0.001
Experience buffer length	10,000
Discount factor	0.95

The desired angles are generated randomly in each episode to prepare the system for any command. The results of agent training are shown in Figure 6.

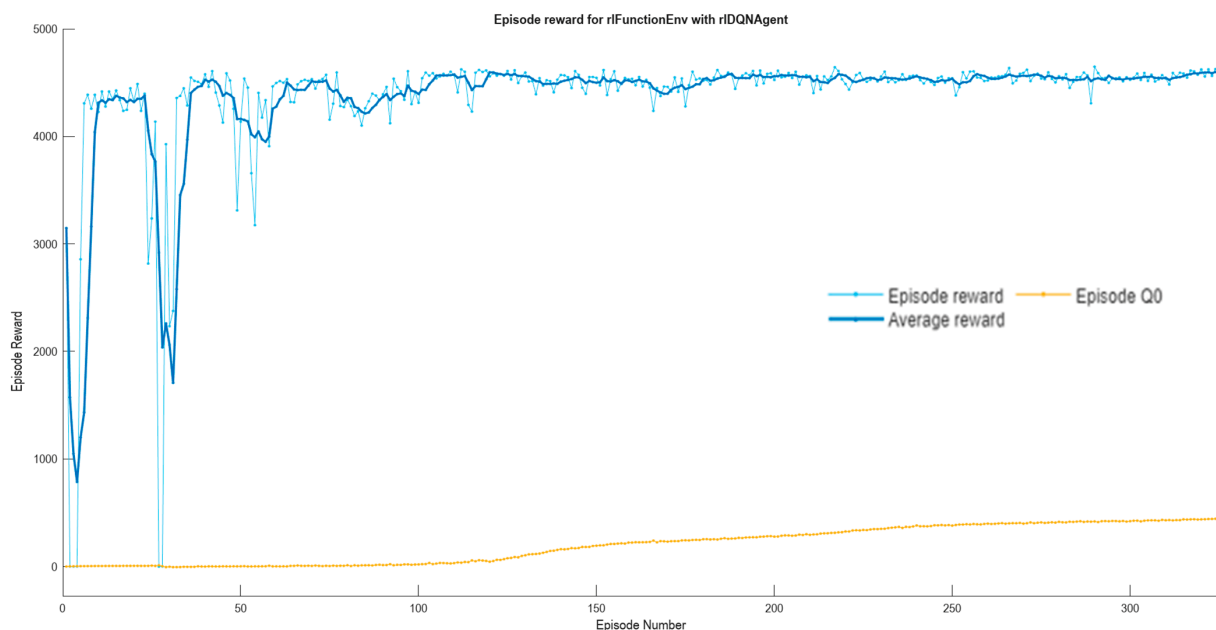


Figure 6. Training result.

As shown in the above-mentioned figure, the agent has successfully learned to achieve high rewards. Additionally, the results of one of the episodes can be seen in Figure 7 below.

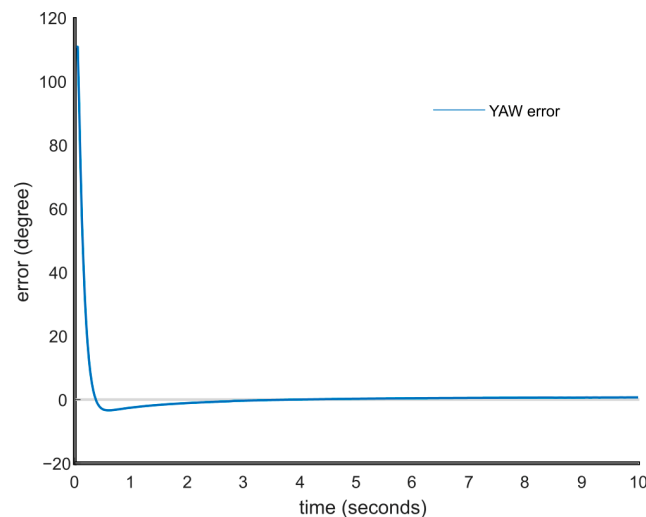


Figure 7. Simulation results in DDQN method.

As shown in Figure 7, the DDQN agent successfully controlled the system and rotated it by 110 degrees. It is important to note that disturbances were added to the system. However, these plots will differ from a hardware implementation due to physical limitations, such as battery voltage drop or DC motors not achieving the desired angular velocity.

To better evaluate whether this is an effective controller for the system, the next section will apply a PID controller and compare the results.

4.2. PID Control Simulation

Using the PID method allows for a comparison with the DDQN method to evaluate its performance. As mentioned in the previous section, the Cohen–Coon method was used for tuning the PID controller. The results of this method were compared with other experimental methods to determine the optimal control method.

The PID controller was less effective in handling disturbances. The results of the PID controller are shown in Figure 8.

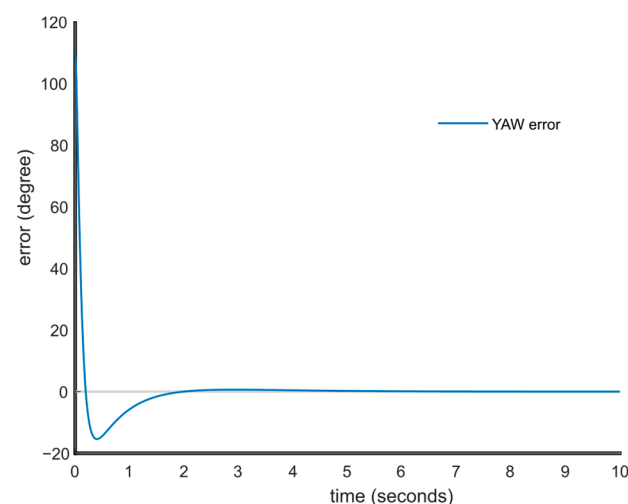


Figure 8. Simulation results in PID method.

Figure 8 shows that the PID controller aims to reduce the 110-degree error to zero, similar to the DDQN method. However, the differences between these two methods indicate that DDQN controlled the system in a shorter time with less overshoot.

In the simulation section, the superior performance of DDQN has been evaluated. Moving forward, these methods will be implemented on hardware, and their performance will be compared in the Real-Time Implementation section.

5. Real-Time Implementation

As mentioned in the previous section, the system has been implemented on real hardware in a laboratory environment to evaluate the performance of the controllers and bridge the gap between theoretical and applied reinforcement learning (RL).

As shown in Figure 9, the CubeSat in this research consists of three main sections:

- Section 1: Battery;
- Section 2: DC motor and reaction wheel;
- Section 3: Hardware.

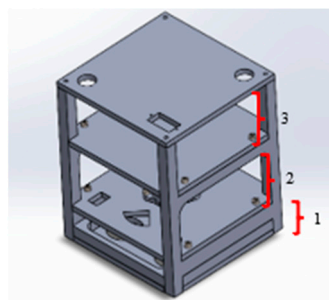


Figure 9. CubeSat sections: 1—battery, 2—DC motor and reaction wheel, 3—hardware.

The hardware components used are described below.

The reaction wheel, mounted on a DC motor, receives control signals from the microcontroller (Figure 9). To ensure robust communication between the microcontroller and the DC motor, an H-bridge motor driver is employed. This driver not only relays control signals from the microcontroller to the motor but also provides voltage regulation and safeguards the motor from electrical faults.

For attitude determination, a gyro sensor is integrated into the system and communicates with the microcontroller via the Universal Asynchronous Receiver-Transmitter (UART) protocol. This sensor captures the CubeSat's orientation at a rate of 100 measurements per second.

Additionally, a wireless NRF module (using the Serial Peripheral Interface (SPI) protocol) enables bidirectional communication between the CubeSat and the ground station for telemetry transmission and command reception.

To power the CubeSat, a 3-cell lithium-ion battery pack (each cell rated at 4 volts, totaling 12 volts) is used. The battery pack is connected to a Battery Management System (BMS) module, which ensures balanced charging/discharging across all three cells and protects against overcharging, over-discharging, and short circuits (Figure 10).

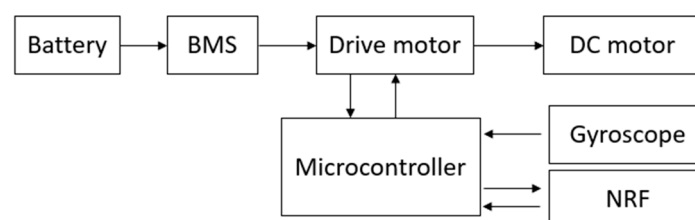


Figure 10. Hardware Diagram.

The control diagram of the system is shown in Figure 11. The process begins when the desired angle is received from the ground station. This reference input is compared with

the actual attitude data from the gyroscope sensor to calculate the error signal. The error is then fed into the controller (either a Deep Learning-based controller or a PID controller), which computes the required angular velocity for the reaction wheel.

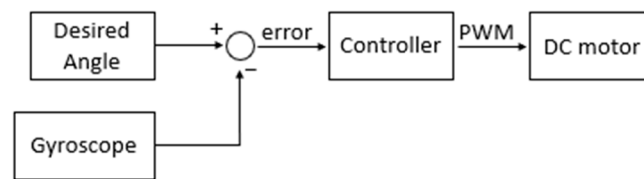


Figure 11. Control Diagram.

The controller outputs this velocity command as a Pulse Width Modulation (PWM) signal, which is sent to the DC motor driving the reaction wheel. This closed-loop control process runs continuously to maintain the CubeSat's desired orientation.

The ground station receives attitude data from the CubeSat and plots the angular values in real time. Additionally, it transmits control commands including Desired attitude targets in DDQN method.

For hardware implementation, the PID coefficients optimized during simulation were directly applied to the controller. The experimental results of this implementation are presented in Figure 12, demonstrating the system's performance under actual flight conditions.

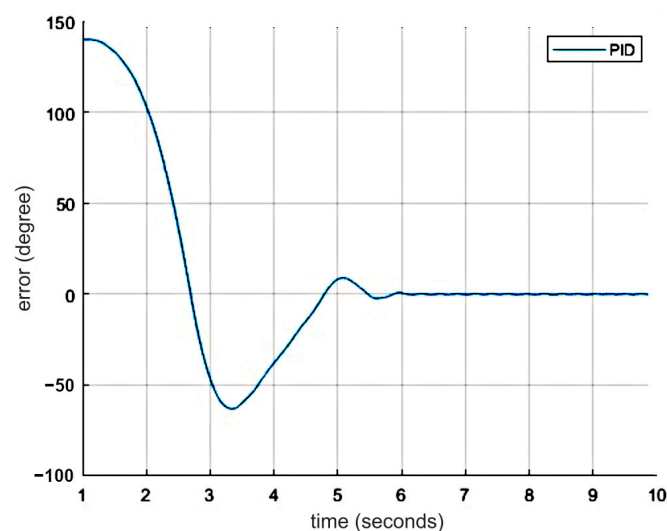


Figure 12. Hardware implementation results in PID method.

Figure 12 shows the performance of the PID controller in the hardware-in-the-loop test. In this experiment, the satellite was commanded to rotate 140 degrees, resulting in an initial error of 140 degrees. The satellite first exhibited significant overshoot, then converged to the desired angle and stabilized. As visible in the plot, even after reaching the target angle, the system shows very minor oscillations, indicating it failed to achieve perfect stability.

To control the system using DDQN, direct implementation of the trained neural network on the microcontroller is not feasible due to its size, as shown in Figure 13. In this setup, the satellite transmits its current state to the ground station. The ground station then processes the data through the neural network and sends the resulting action back to the microcontroller, which executes it on the motor.

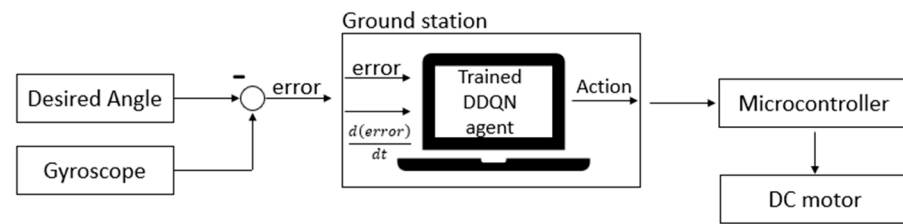


Figure 13. DDQN diagram.

The results of this implementation can be observed in Figure 14.

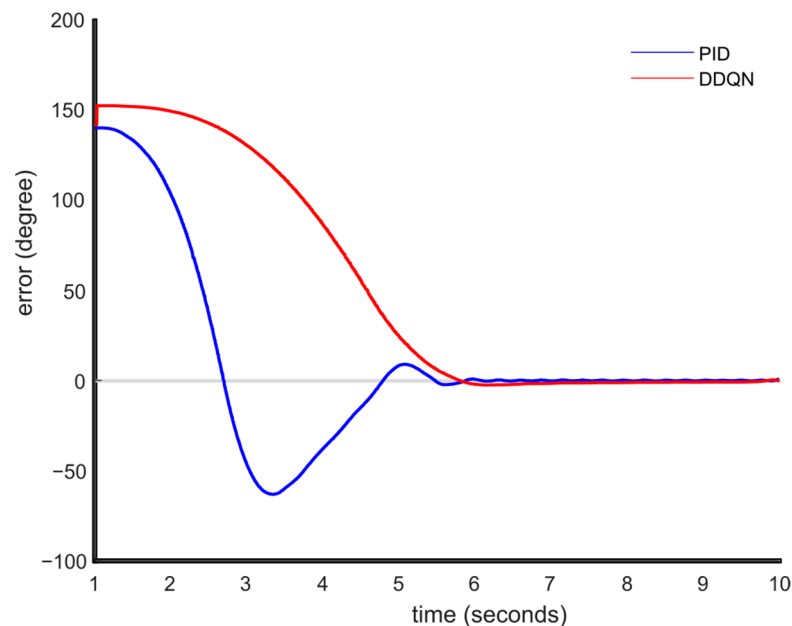


Figure 14. Implementation results of the DDQN method compared with the PID controller.

Figure 14 demonstrates the controller's performance, which exhibits slower response dynamics compared to the PID controller but achieves precise attitude control without overshoot during the 150° rotation maneuver. Key observations include: The controller maintains superior steady-state accuracy relative to the PID implementation, with complete elimination of overshoot phenomena, notably absent are the high-frequency oscillations observed in the PID results during terminal phase operations and Both controllers demonstrate comparable settling times, reaching the target attitude at approximately $t = 6$ s.

A significant divergence exists between simulation predictions and experimental results, which will be analyzed in detail in the following section.

6. Conclusions

This study aimed to design and implement a deep learning-based controller through simulation and compare its performance with classical control methods. Subsequently, utilizing mechatronic expertise and laboratory equipment, we implemented the system under experimental conditions to evaluate its real-world performance.

Our work bridges the gap between reinforcement learning theory and practical implementation, demonstrating the superior capability of deep learning-based controllers in handling systems with unpredictable disturbances. As discussed in the previous chapter, the deep learning controller outperformed classical controllers in both simulation and hardware implementation. During hardware testing, the PID controller exhibited significant overshoot when facing disturbances, while the deep learning controller maintained smooth operation without overshoot or noise, despite natural system disturbances. Furthermore,

the relative instability observed in the final phase of PID implementation was absent in the deep learning controller, clearly indicating its better performance.

As noted in earlier chapters, notable differences exist between simulation and hardware implementation results. For instance, while the deep learning controller stabilized the system in under 2 s during simulation, the hardware implementation required 6 s. This discrepancy stems from several key factors:

1. Motor dead zone: Simulations allow application of minimal angular velocities, whereas practical implementation faces physical limitations. The motor's dead zone prevents movement below a certain PWM threshold (e.g., 30), significantly impacting real-world performance.
2. Battery voltage drops: The quality of batteries used in hardware implementation critically affects performance. Voltage drops can prevent the motor from reaching desired speeds.
3. Test platform disturbances: Although simulated in code, actual experimental conditions may introduce larger disturbances than anticipated, prolonging stabilization time.

This research successfully implemented a deep learning-trained controller on hardware. Future work could explore:

- Three-degree-of-freedom CubeSats with three reaction wheels;
- Alternative actuator configurations beyond reaction wheels;
- Comparative studies of other reinforcement learning algorithms;
- Implementation on different satellite platforms.

The findings highlight the potential of deep learning approaches for attitude control systems while emphasizing the importance of accounting for hardware limitations in control system design.

Author Contributions: Conceptualization, J.R. and M.M.; methodology, S.Z., J.R., M.M. and K.G.; software, J.R.; validation, J.R. and M.M.; resources, M.M.; writing—original draft preparation, S.Z.; writing—review and editing, J.R., M.M. and K.G.; visualization, S.Z.; supervision, J.R.; project administration, M.M. and K.G.; funding acquisition, K.G. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by European Regional Development Fund under “Research Innovation and Digitization for Smart Transformation” program 2021–2027 under the Project BG16RFPR002-1.014-0006 “National Centre of Excellence Mechatronics and Clean Technologies”.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data will be available on request.

Acknowledgments: The author K. Georgiev is thankful for the support provided by the European Regional Development Fund within the Operational Programme “Bulgarian national recovery and resilience plan”, procedure for direct provision of grants “Establishing of a network of research higher education institutions in Bulgaria”, and under Project BG-RRP-2.004-0005 “Improving the research capacity and quality to achieve international recognition and resilience of TU-Sofia (IDEAS)”.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Krishna, N.S.; Gosavi, S.; Singh, S.; Saxena, N.; Kailaje, A.; Datla, V.; Shah, P. Design and implementation of a reaction wheel system for CubeSats. In Proceedings of the 2018 IEEE Aerospace Conference, Big Sky, MT, USA, 3–10 March 2018; pp. 1–7.
2. Li, J.; Post, M.; Wright, T.; Lee, R. Design of Attitude Control Systems for CubeSat-Class Nanosatellite. *J. Control. Sci. Eng.* **2013**, *2013*, 657182. [[CrossRef](#)]

3. Ge, S.; Cheng, H. A comparative design of satellite attitude control system with reaction wheel. In Proceedings of the First NASA/ESA Conference on Adaptive Hardware and Systems (AHS'06), Istanbul, Turkey, 15–18 June 2006; pp. 359–364.
4. Chiang, R.Y.; Jang, J.-S.R. Fuzzy logic attitude control for Cassini spacecraft. In Proceedings of the 1994 IEEE 3rd International Fuzzy Systems Conference, Orlando, FL, USA, 26–29 June 1994; pp. 1532–1537.
5. Karr, C.L.; Freeman, L.M. Genetic-algorithm-based fuzzy control of spacecraft autonomous rendezvous. *Eng. Appl. Artif. Intell.* **1997**, *10*, 293–300. [[CrossRef](#)]
6. Chaurais, J.R.; Ferreira, H.C.; Ishihara, J.Y.; Borges, R.A. Attitude control of an underactuated satellite using two reaction wheels. *J. Guid. Control. Dyn.* **2015**, *38*, 2010–2018. [[CrossRef](#)]
7. Sutton, R.S.; Barto, A.G. *Reinforcement Learning: An Introduction*, 2nd ed.; The MIT Press: London, UK, 2018.
8. Yadava, D.; Hosangadi, R.; Krishna, S.; Paliwal, P.; Jain, A. Attitude control of a nanosatellite system using reinforcement learning and neural networks. In Proceedings of the 2018 IEEE Aerospace Conference, Big Sky, MT, USA, 3–10 March 2018; pp. 1–8.
9. Vedant, J.T. Reinforcement learning for spacecraft attitude control. In Proceedings of the 70th International Astronautical Congress, Washington, DC, USA, 21–25 October 2019.
10. Tammam, A.; Aouf, N. Hierarchical Deep Reinforcement Learning for cubesat guidance and control. *Control. Eng. Pract.* **2025**, *156*, 106213. [[CrossRef](#)]
11. Kuroiwa, S.; Kogiso, N. Resilient Operation Planning for CubeSat Using Reinforcement Learning. In Proceedings of the PHM Society Asia-Pacific Conference, Tokyo, Japan, 11–14 September 2023.
12. Ramezani, M.; Alandihallaj, M.A.; Sanchez-Lopez, J.L.; Hein, A. Safe Hierarchical Reinforcement Learning for CubeSat Task Scheduling Based on Energy Consumption. *arXiv* **2023**, arXiv:2309.12004. [[CrossRef](#)]
13. Mirshams, M. CubeSat's Hands-on Training Package (CHTP). In Proceedings of the 43rd COSPAR Scientific Assembly, Sydney, Australia, 28 January–4 February 2021.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.